

The goal of this task was to create a model for finding the best possible lineup for an NHL team (here chosen as the Vegas Golden Knights). Given the fact that there are many sources of data available from public APIs, my first thought was to carefully define the statement "best possible lineup." For simplicity, my first idea was to choose just one good metric that would reflect the current or past performance of a player, with the possibility of adding more variables later. The total amount of points (goals + assists) scored in past games seemed to be the most straightforward way to represent the performance of any single player, and therefore it was chosen as the metric.

Since the situation of play (even strength, power play, short-handed) has a major impact on the dynamics of play, I decided to also use points scored in power play and short-handed situations to identify the players who are performing best in these particular scenarios. The big picture of the model was then to rank the players based on performance in the N previous games and select lineups for even strength, power play, and short-handed play accordingly.

I chose to use the NHL public API at <https://api-web.nhle.com> for simple access to the relevant data. Python was chosen as the programming language for its easy-to-use libraries (and because it is my language of choice).

Instructions for running the code are provided in the README.md file. All the functionalities are also explained there as well as in comments throughout the code. The overview is as follows. First, we have dataclasses that store the data from the API calls (models.py). The fetcher.py file then fetches data (team roster and season game log for every player) from the NHL API, stores the data using the aforementioned classes, and calculates summaries from the N past games for each player. The engine.py file then builds the lineups according to situation. For power play and short-handed play, it outputs only one unit (5 players), whereas for even strength we get 3 units ordered by the chosen metric. The Uvicorn library is used to display the results in the browser with easy use of curl.

Possible additions to further develop this model is to add other metrics, for example, to identify best short-handed players since only points scored in short-handed situations might not be reflective of good performance in this instance. Such variables could include, goals scored by opponent, penalties in minutes (to prevent 5vs3) and defensive metrics.